

Dynamic Model Routing: Demand and Open-Source Relevance

Assessment

There is credible demand for better model routing, but the evidence supports a focused utility for experienced developers rather than a broad new routing platform. A free GitHub release could be useful if it makes an existing workflow easier to configure, inspect, and verify. Free distribution, multiple supported clients, and risk-aware routing are already available elsewhere; they are not sufficient differentiation.

Developers request task-dependent model selection, cheaper subagents, and preservation of intentional choices. Some also object to expensive-coordinator overhead when dispatching cheap work. These are real workflow problems; public requests do not establish adoption of another plugin.^{1, 2, 3}

Competition changes the investment decision. Claude Code Harness publishes a multi-host routing policy, Superpowers + GStack includes domain-sensitive model selection, and Codex-specific routing skills already preserve the coordinator while delegating bounded work. A new implementation must justify its additional installation and maintenance burden against these alternatives.^{14, 15, 17}

Recommendation: narrow the project and validate before implementing the full six-surface release. Package the supplied routing policy as a small, independent routing kit, initially tested on Codex CLI and Claude Code CLI. Keep the shared policy portable, but expand the supported-surface claim only when repeat users and runtime evidence justify it. Consider contributing a minimal routing mode to an existing project if its architecture can accommodate the same outcome with less maintenance.

The strongest proposed positioning is: **"A small, inspectable routing policy for coding agents, with explicit model choices, bounded escalation, and evidence of what actually ran."** This is a hypothesis about a useful package, not a claim of a unique algorithm or guaranteed savings. Its practical advantage must be demonstrated through setup experience, compatibility, and completed-task results.

Decision	Assessment
Do developers experience the problem?	Yes: direct requests and reports across several communities
Do they need this exact plugin?	Unproven: the most important remaining validation question
Is the generic idea novel?	No: significant native and open-source overlap
Can a free release remain relevant?	Yes, conditionally: simplicity, compatibility, and credible evidence
Is a large universal router justified now?	No: start with a bounded pilot and explicit expansion criteria

Evidence is current as of 12 September 2026. Adoption and savings remain unproven.

1. saidelike / OpenAI #9205
2. quarrel / OpenAI #32961
3. jagalliers / Anthropic #81050
14. Chachamaru127: Model Routing Policy
15. Paretofilm: Superpowers + GStack: Model Routing Table
17. orange-the-weak: Codex Auto Model Router

Demand evidence

The most useful signals describe a specific inconvenience or failure. They are stronger than generic enthusiasm for AI, but weaker than sustained use of a working release. The following examples deliberately include historical and partially satisfied requests so that existing product improvements are not mistaken for an open market gap.

Primary source	Expressed problem	Relevance and limitation
Codex #9205, 14 Jan 2026	Use different models for reasoning, implementation, and tool-heavy work	Direct demand for task routing; later native subagent controls partly address it. ¹
Codex #32961, 14 Jul 2026	Bind a model and effort to a skill without an expensive parent first delegating it	Direct demand for lower-friction selection; a subagent-based plugin does not fully solve the request. ²
Claude Code #81050, 25 Jul 2026	Express relative worker defaults, spending ceilings, and capability floors while retaining intentional choices	Strong residual policy demand; a prompt policy cannot guarantee host-level enforcement. ³
Claude Code #68147, 13 Jun 2026	A requested subagent model reportedly changed after continuation	Historical reliability evidence. The issue is closed; the report is not proof of a current defect. ⁴
r/codex, page displaying 28 days ago	Coordinate subscribed coding CLIs while avoiding expensive boilerplate work and repeated failure loops	Supports the underlying need, but asks for cross-CLI orchestration beyond a plugin operating separately in each client. ⁵
r/cursor, page displaying about one month ago	Unexpected premium-model use in Auto mode	Anecdotal demand for predictability. Replies identify the native Cost setting as an alternative. ⁶

Three distinct needs emerge. First, developers want to avoid spending premium capacity on routine work. Second, they want model selection to fit into their existing workflow without constant intervention. Third, they want selected models and policy constraints to persist and remain visible. A routing classifier addresses only part of this collection of needs.

The distinction matters for the proposed product. A developer asking to eliminate coordinator overhead may reject a plugin whose main technique adds a coordinator and reviewer. A developer asking for handoffs among subscriptions may reject a plugin that has no handoff mechanism. Recruitment and evaluation should ask which outcome matters before counting either person as a likely adopter.

Public discussions overrepresent people with unusually complex setups, dissatisfaction, or products to promote. Replies advertising an author's own router are supply evidence, not independent adoption evidence. No denominator or representative sample is available, and a closed issue should not be presented as an unresolved opportunity. Interviews and repeated real-project use remain necessary to establish product demand.

1. saidelike / OpenAI #9205
2. quarrel / OpenAI #32961
3. jagalliers / Anthropic #81050
4. Necmttn / Anthropic #68147
5. InkFasten / r/codex: Anyone successfully routing between Claude Code, Codex, Grok, and other CLIs while staying on subscriptions?
6. TheRealAniiXx and respondents / r/cursor: Something about the model auto-routing changed (allegedly)

Native alternatives

The default competitor is the developer's existing coding tool. A new plugin must improve on native settings and a short project instruction, not only on an expensive model used indiscriminately. This is a higher and more useful benchmark.

Host	Native baseline	Implication for a separate plugin
Cursor	Auto routes requests by task type and complexity, with Cost, Balance, and Intelligence modes; per-request routing is not user-configurable	Generic automatic cost/quality selection is already covered. The remaining hypothesis is explicit project policy and evidence. ⁷
Claude Code	Model selection, simple-model subagents, context management, and effort reduction are documented cost controls	Much of the workflow can be assembled without a new plugin. ⁹
Codex	Subagents can use explicit model/effort pairs or project roles	A plugin can supply consistent decisions and setup, but native dispatch is the execution mechanism. ¹⁰

Cursor reports that its router was launched on 22 July 2026 and describes training from production interactions, including model-switching cache effects. Its published savings and satisfaction results are vendor claims under its own evaluation method. They demonstrate a serious, improving native alternative; they do not predict what a small external policy will achieve.⁸

Capabilities also differ within the proposed support matrix. Claude Code recognizes a skill's model metadata for a turn, while that is not a portable promise of ordinary skill text. Claude Desktop Code has an interactive interface and separate deployment limitations. Cursor can substitute an available model when configured choices are restricted. Native behavior must be respected and tested rather than hidden behind a universal-looking command.^{11, 22, 23}

The residual opportunity is therefore not "make models selectable." It is a maintained, inspectable decision procedure that helps a developer choose appropriate native controls and verify the outcome. That can include explaining why a tiny authorization change needs a strong route, preserving a deliberate model pin, or showing that runtime identity was unavailable. It can also recommend staying in the current model when delegation would waste time.

The product should make its control boundary visible. Advisory policy, a successfully requested native dispatch, and a confirmed effective model are different states. A user requiring an inviolable spending ceiling or a particular model for every inference needs a host-level enforcement mechanism. A skill should not claim to provide that guarantee merely because it contains a prohibition.

Assessment: native improvements reduce the value of generic routing, but can increase the usefulness of a small compatibility layer that accurately configures and checks them. That opportunity lasts only while the layer removes more friction than it adds.

7. Cursor: Cursor Router
8. Connor O'Keefe and Yuri Volkov / Cursor: How Cursor Router chooses the right model for the task
9. Anthropic: Manage costs effectively
10. OpenAI: Subagents
11. Anthropic: Extend Claude with skills
22. Anthropic: Desktop application
23. Cursor: Subagents

Open-source competition

The closest alternatives include workflow frameworks and compact routing skills, not just API gateways. Repository descriptions establish intended functionality and public availability. They do not independently establish reliability or performance.

Alternative	Documented overlap	Relevant difference
Claude Code Harness	Multi-host model/effort roles, explicit override priority, independent review and completion evidence	Broad delivery framework; a smaller routing-only package could appeal to developers who already have a workflow. ^{13, 14}
Superpowers + GStack	Per-skill tiers modified by domain sensitivity, native Claude subagents	Routing is explicitly advisory and not a benchmarked matrix; other hosts need their own mappings. ¹⁵
capitalparser/codex-model-router	Bounded Codex workers, risk/verification factors, observed outcomes and escalation	Codex-specific, with a Python advisor and model-specific roles. ¹⁶
orange-the-weak/codex-auto-model-router	Skill-first native worker selection, preserved coordinator, overhead-based delegation decision	Codex-focused; already occupies much of the proposed lightweight shape. ¹⁷
Claude Code Router	Local gateway for multiple coding clients, conditional routes, fallbacks, logs and provider management	More operational machinery and credential configuration than a native skill; a better fit for centralized request control. ¹⁸

The strongest direct competitor is Claude Code Harness. Its routing policy, updated 5 September 2026, names Claude Code, Codex, Cursor, and Grok. It also acknowledges that plan and administrative restrictions can cause native fallback. This is substantial overlap with a cross-platform policy that respects explicit choice and distinguishes recommendation from execution.¹⁴

Risk-aware routing itself is also occupied. Superpowers + GStack separates cognitive demand from domain sensitivity, so a mechanically small task in a high-consequence domain can require a stronger model. This means the original skill's risk discipline is valuable, but cannot honestly be marketed as a new category by itself.¹⁵

GitHub displayed approximately 37.2k stars for Claude Code Router, 3.1k for Claude Code Harness, and 48 for Codex Auto Model Router on the reviewed pages. These snapshots show very different levels of repository attention; they are not active-user counts, install counts, or evidence that any project is superior. A larger project's interest may come from features unrelated to routing.^{18, 13, 17}

Competitive judgment: "free, cross-platform, skill-based, and safety-conscious" is an insufficient launch message. A useful new project must demonstrate a materially smaller setup, compatibility with existing workflows, and more trustworthy execution evidence. If an existing project can offer those qualities through a small contribution, building a separate maintenance obligation is difficult to justify.

13. Chachamaru127: Claude Code Harness
14. Chachamaru127: Model Routing Policy
15. Paretofilm: Superpowers + GStack: Model Routing Table
16. capitalparser: Codex Model Router
17. orange-the-weak: Codex Auto Model Router
18. musistudio: Claude Code Router

Audience and product fit

The likely early audience is experienced developers who already understand model tradeoffs and use coding agents regularly. They can recognize routing mistakes, supply meaningful acceptance checks, and explain when setup overhead becomes a burden. This is an analytical segment hypothesis, not a measured customer distribution.

Segment	Likely value	Main reason to decline
Developers using two or more coding clients	One understandable policy across existing workflows	May actually need context handoff or cross-CLI execution, which is a different product
Small teams with repeated review and quality requirements	Shared route definitions and consistent evidence records	An existing workflow framework or a short repository policy may suffice
Single-client power users with recurring model-selection friction	Easier setup, predictable defaults, and visible decisions	Native settings may provide the same outcome with less overhead
Occasional users satisfied with Auto/defaults	Limited likely benefit	Another plugin creates decisions and maintenance instead of removing them

The first two groups deserve a pilot, but they should not be combined into one vague "all developers" target. A solo developer optimizing included subscription capacity has different success criteria from a team enforcing review boundaries. A user who wants arbitrary cross-provider execution needs more than portable native adapters. Each interview should establish the actual workflow before presenting features.

The proposed policy already contains useful product material: Advise, Apply, and Configure modes; four stable risk routes; limited retries; independent review for sensitive work; and cost measured only when reliable evidence is exposed. Those mechanisms can be shared as a useful reference even if a larger plugin is never justified.²¹

The strongest product hypothesis is **less configuration and less disruption**, supported by transparent results. Keep model identifiers and effort support in small host-specific bindings. Let the routing policy work alongside existing AGENTS.md or CLAUDE.md conventions rather than introducing a compulsory planning system. Avoid asking users to replace their workflow just to select a worker model.

Three interpretations of "platform agnostic" should remain separate. A common policy can be portable. Packaging and dispatch require platform-specific adapters. Moving work, permissions, and context between different clients is cross-platform orchestration. Success at the first two does not imply the third, and some of the strongest public demand explicitly concerns that third problem.⁵

The benefit that would justify installation is concrete: after a short setup, a recurring class of work finishes with fewer interventions or lower total resource use, while its existing acceptance checks still pass. A clear explanation of a route is useful; it becomes a product only if people repeatedly prefer using it to their previous method.

5. InkFasten / r/codex: Anyone successfully routing between Claude Code, Codex, Grok, and other CLIs while staying on subscriptions?
21. Supplied Dynamic Model Routing policy; private source.

Cost, quality, and proof of value

A cheaper model does not automatically produce a cheaper completed task. Routing adds classification, context transfer, worker startup, result aggregation, and sometimes independent review. Additional failures or repeated context loading can erase a token-price advantage. The Codex request for model-bound skills is particularly useful counterevidence because it explicitly objects to this delegation overhead.²

A practical comparison should count all work needed to reach acceptance: coordinator usage, workers, reviews, retries, tool costs where relevant, and developer intervention. It should also record wall-clock time. These dimensions should remain separate rather than being compressed into an unexplained "efficiency score." A routing policy that saves tokens but requires more human repair may be a poor trade.

The most useful primary metric is **total measured cost per accepted task** where billing-equivalent measurements exist. Where subscription billing does not expose task-local dollars, report available usage and completion outcomes without translating them into invented monetary savings. Lower consumption may preserve capacity while leaving the subscription bill unchanged. Shared usage limits are not universally fixed by switching to another model.⁹

Quality needs a separate baseline. The 2025 Stack Overflow survey reported that 66% of respondents to the frustration question encountered almost-correct AI solutions and 45% found debugging AI-generated code more time-consuming. These results support the importance of rework and verification, not demand for this particular plugin. The survey population and question denominators should not be treated as the market for routing tools.¹⁹

Dimension	Suitable observation	Misleading substitute
Cost	Entire accepted-task usage, with pricing assumptions recorded	Cheap worker output tokens alone
Speed	End-to-end elapsed time and developer wait time	A model's advertised generation speed
Quality	Existing acceptance checks, independent review, and escaped defects	Confidence, agreement, or one successful build
Routing accuracy	Requested and effective identity when the host exposes it	A skill announcing a model name
Adoption	Repeated use on real work after the first trial	Stars, page views, or a compliment

The pilot should compare three realistic baselines: the participant's normal model, native Auto/default behavior where available, and a short manual routing policy without the plugin. A relevant existing open-source router should be included when the participant already uses one. This tests the value of the package, not merely whether cheaper models sometimes work.

No savings claim is justified yet. Report successful and failed task classes, including cases where routing costs more. The ability to identify when not to route may become one of the project's most useful features.

2. quarrel / OpenAI #32961
9. Anthropic: Manage costs effectively
19. Stack Overflow: 2025 Developer Survey: AI

Relevance as a free GitHub project

Free distribution removes a purchase decision, but not the costs of evaluating trust, installing components, configuring models, and keeping integrations current. It also does not supply free model inference. A useful repository earns continued use by reducing those costs and providing credible results.

A standalone release can have several kinds of value. It may become a practical utility for a small group, a reference implementation for safer delegation, or a reusable set of compatibility tests for other tools. These outcomes do not require a large business or a large star count. The relevant question is whether its maintenance cost is proportionate to demonstrated repeat use or downstream reuse.

The release should be understandable in one sitting. Its README should state the supported clients and versions, the exact activation modes, a short realistic example, and the distinction between recommended and effective routing. It should explain that the main conversation is not universally switched by skill prose and that cross-CLI handoff is outside the initial scope. Installation and removal should have equally clear instructions.

An explicit license is necessary to make intended reuse clear; public visibility alone is not the same as broad permission to modify and redistribute. GitHub's licensing guidance explains this distinction. The choice of license should be deliberate and should account for any reused third-party material.²⁰

The strongest launch evidence would be a small set of reproducible task comparisons, a compatibility table with dated results, and several independently written accounts of continued use. Do not substitute a large model ranking table or a claimed savings percentage without the underlying workload. A release that clearly admits unsupported controls is more credible than one that claims seamless behavior on six surfaces after checking only manifest syntax.

Distribution should follow the audience. A documented GitHub release provides the canonical source; native marketplace packaging can reduce installation friction on clients where it is supported. Discussion posts should explain the specific gap and include an honest comparison with existing tools. Treat community responses as feedback, not permission to contact people or publish on their behalf.

There is a viable contribution path as well. Before maintaining a new framework, determine whether a routing-only mode, additional host adapter, or stronger execution-evidence test would be accepted by a relevant existing project. The repository could still preserve the original policy and research while implementation effort goes where it will be used. This is an option to evaluate, not an assumption that maintainers will accept a contribution.

20. GitHub: Licensing a repository

Validation plan

Run a small pilot before promising full desktop and CLI coverage. The following thresholds are proposed product decisions, not industry benchmarks or observed results. Their purpose is to prevent an appealing concept from expanding before there is evidence of usefulness.

Recruit eight developers who already use Codex or Claude Code regularly, including several who use both. Ask about a recent task: model selection, switching effort, and failure recovery. Include developers satisfied with native defaults. Avoid asking only whether automatic routing sounds useful.

First compare the supplied policy, native controls, and one close alternative on a short task. If people can achieve the same outcome with a paragraph of instructions, publish the skill and examples rather than building additional machinery. Only participants with recurring friction should enter a plugin trial.

For the trial, use two weeks of real work and a modest matched task set. Target at least twenty comparable accepted-task observations across five or more participants, with enough repeat use to separate initial curiosity from habit. Use fresh equivalent branches and vary evaluation order where practical. Keep acceptance checks stable and do not encourage sensitive changes merely to enlarge the sample.

Gate	Proposed criterion	Decision if missed
Activation	At least 6 of 8 participants configure the first route within 10 minutes without author intervention	Simplify setup before adding hosts
Repeat use	At least 5 use it in a second week on real tasks	Reassess demand; a reference skill may be enough
Practical advantage	At least 3 of the minimum 5 retained users pass the preselected metric and guardrails below	Narrow the task class or stop the plugin
Integrity	No observed unauthorized substitution, out-of-scope action, or omitted required acceptance check	Pause rollout and diagnose the failure
Additional host	At least 3 retained participants request that host and can help verify it	Keep the adapter provisional

Zero observed integrity failures in a small sample is a release gate, not statistical proof of safety. Report unknown model identity rather than counting it as success. If host evidence is absent, distinguish behavioral compliance from actual enforcement and keep hard guarantees out of the product description.

Before the trial, assign each task class one primary metric. For a cost cohort, require at least 15% lower total measured cost, or a prespecified usage measure when cost is unavailable. For a workflow cohort, require at least 25% less hands-on switching and repair time. Keep acceptance unchanged; allow no more than 10% worse median elapsed time, and no more than 10% extra measured cost in the workflow cohort. Report unavailable guardrails as unverified, not passed. Do not choose a winning metric retrospectively.

This pilot should produce a decision record: continue the small standalone project, narrow it to one workflow, contribute to an existing tool, or stop at a published reference policy. Each is a legitimate outcome.

Durability and final recommendation

The main long-term risk is native substitution: coding platforms can add configurable defaults, stronger automatic routing, or improved model-bound skills. The second risk is integration drift. A plugin that supports three clients across desktop and CLI inherits differences in installation, configuration precedence, permission handling, and runtime visibility. Model names change more quickly than the underlying principle of matching risk to capability.^{10, 11, 12, 22, 23}

Maintain the stable policy separately from host details. Record tested versions, keep adapters small, and retire compatibility work that has no active users. Refresh model bindings from real availability rather than treating old rankings as permanent. Treat a platform regression as a compatibility problem; do not promise that a prompt can repair it.

There are three plausible durability scenarios. If native tools cover both selection and inspection well, the project may shrink into documentation and tests. If native tools remain capable but inconsistent, a small shared policy and compatibility kit can remain useful. If demand concentrates on cross-client execution and hard spending enforcement, the need has shifted toward a different, more operational product with existing competitors.

The supplied skill has a sound organizing idea: optimize the entire path to an accepted result while preserving constraints. That makes it a worthwhile asset to share. Its additional value as a maintained plugin depends on evidence that packaging and adapters improve real workflows. The difference between a useful skill and a valuable integration is repeated use with less friction.²¹

Proceed with a small routing-only pilot; make public release conditional on its activation, repeat-use, and practical-advantage gates. Prioritize Codex CLI and Claude Code CLI because they allow model-aware workers and align with the demand signals. A trial needs only the smallest working policy/adaptor artifact. Keep the policy portable and defer six-surface support until the relevant users and tests exist.

The strongest justification for an independent project would be a compact setup that works alongside existing workflows, transparent native capability reporting, and measured benefits on a clearly described task class. The strongest reason to stop would be that participants prefer native controls or an existing router after a fair trial. If the only distinguishing feature is a different set of model names, the project does not merit ongoing integration work.

Answer to the central question: some developers clearly need a better way to coordinate cost, capability, and verification. There is not yet evidence that they need this particular plugin. A free GitHub release can be relevant as a focused, trustworthy utility and reference, provided its advantage is demonstrated and its scope stays small.

Evidence limitations remain material. Public requests are self-selected, maintainer descriptions are not independent benchmarks, issue status can change, and no retention or completed-task data exists for this proposed package. The recommendation is therefore a bounded investment decision, not a prediction of adoption.

10. OpenAI: Subagents

11. Anthropic: Extend Claude with skills

12. Anthropic: Create custom subagents

21. Supplied Dynamic Model Routing policy; private source.

22. Anthropic: Desktop application

23. Cursor: Subagents

Sources

Sources without a publication date are living documents or repository pages reviewed on 12 September 2026. Repository metrics are approximate displayed snapshots, not usage measurements. Relative Reddit dates are preserved rather than converted into unsupported exact dates. The supplied policy is a design source, not market-demand evidence.

1. saidelike / OpenAI. [Codex could automatically use different models based on task type, issue #9205](#). 14 January 2026. Primary feature request; historical task-routing demand.
2. quarrel / OpenAI. [Allow skills or user commands to declare the model used for their invocation, issue #32961](#). 14 July 2026. Primary request; manual-switching and delegation-overhead concerns.
3. jagalliers / Anthropic. [Subagent model policy: relative bias and cap/floor bounds, issue #81050](#). 25 July 2026. Primary policy request; proposed controls are not shipping features.
4. Necmtn / Anthropic. [Subagent model override silently dropped after a continuation boundary, issue #68147](#). 13 June 2026; closed when reviewed. Historical user-reported reliability issue; not independently reproduced.
5. InkFasten / r/codex. [Anyone successfully routing between Claude Code, Codex, Grok, and other CLIs while staying on subscriptions?](#). Page displayed 28 days ago. First-person demand with broader cross-CLI scope.
6. TheRealAniiXx and respondents / r/cursor. [Something about the model auto-routing changed \(allegedly\)](#). Page displayed about one month ago. Anecdotal concern and native-control counterevidence.
7. Cursor. [Cursor Router](#). Living product documentation. Task-based native Auto and optimization modes.
8. Connor O'Keefe and Yuri Volkov / Cursor. [How Cursor Router chooses the right model for the task](#). 6 August 2026. Vendor methodology and performance claims; no independent validation implied.
9. Anthropic. [Manage costs effectively](#). Living documentation. Model/context controls, subagents, usage and limit distinctions.
10. OpenAI. [Subagents](#). Living documentation. Native worker models, effort, roles, and inherited controls.
11. Anthropic. [Extend Claude with skills](#). Living documentation. Native skill metadata and turn-scoped model override.
12. Anthropic. [Create custom subagents](#). Living documentation. Model selection, precedence, native fallback and runtime inspection.
13. Chachamaru127. [Claude Code Harness](#). Repository README and displayed metrics. Workflow scope and public attention; maintainer claims.
14. Chachamaru127. [Model Routing Policy](#). Updated 5 September 2026. Multi-host role routing and override policy; maintainer documentation.
15. Paretofilm. [Superpowers + GStack: Model Routing Table](#). Version 0.2 notes dated 4 July 2026. Advisory task/domain routing; explicitly not a benchmarked matrix.
16. capitalparser. [Codex Model Router](#). Repository README. Bounded native workers, deterministic advice and verification-aware routing; maintainer claims.
17. orange-the-weak. [Codex Auto Model Router](#). Repository README and displayed metrics. Skill-first routing, overhead handling and installation; maintainer claims.
18. musistudio. [Claude Code Router](#). Repository README and displayed metrics. Local gateway, provider configuration, routing and logs; maintainer claims.
19. Stack Overflow. [2025 Developer Survey: AI](#). 2025. AI-frustration question had 31,476 responses; contextual quality evidence, not a routing market estimate.
20. GitHub. [Licensing a repository](#). Living documentation. Public visibility and licensing distinction.
21. Original author. Dynamic Model Routing skill, Codex setup reference, and metadata. Supplied local source, undated, reviewed 12 September 2026; no public URL supplied. Foundation policy and constraints.
22. Anthropic. [Desktop application](#). Living documentation. Code-tab behavior, shared configuration, and surface-specific limitations.
23. Cursor. [Subagents](#). Living documentation. Editor/CLI support, model configuration and native fallback.